

# Tech Nation Visa - Exceptional Promise

**Criteria:** Mandatory Criteria (Innovation) & Optional Criteria 3 (Product Impact)

**Applicant:** Odeyemi Olusegun Israel

---

## Evidence 8: Flutter Consumer App & Web3 Integration

### 1. Cross-Platform Consumer Application

Alongside the enterprise compliance dashboard, I engineered an end-user facing application using **Flutter (Dart)**, targeting all six platforms (iOS, Android, Web, macOS, Windows, Linux) from a single codebase. This mobile-first app acts as the consumer gateway for initiating and approving blockchain transactions that are silently routed through the AMTTP compliance engine.

The Flutter app uses **Riverpod** for state management, **go\_router** for navigation, and raw **dart:js\_util** interop for direct Web3 wallet connectivity — proving advanced cross-platform engineering that bridges native mobile frameworks with decentralised blockchain protocols.

#### Flutter App Architecture:

- **Framework:** Flutter (Dart) — single codebase targeting iOS, Android, Web, macOS, Windows, Linux
- **State Management:** Riverpod exclusively (ConsumerWidget pattern)
- **Navigation:** go\_router with role-based route guards (R1–R2)
- **Web3 Integration:** Raw `dart:js_util` interop for MetaMask/WalletConnect
- **Key Screens:** Home, Wallet Connect, Send Transaction, Transaction History, Trust Check, Disputes

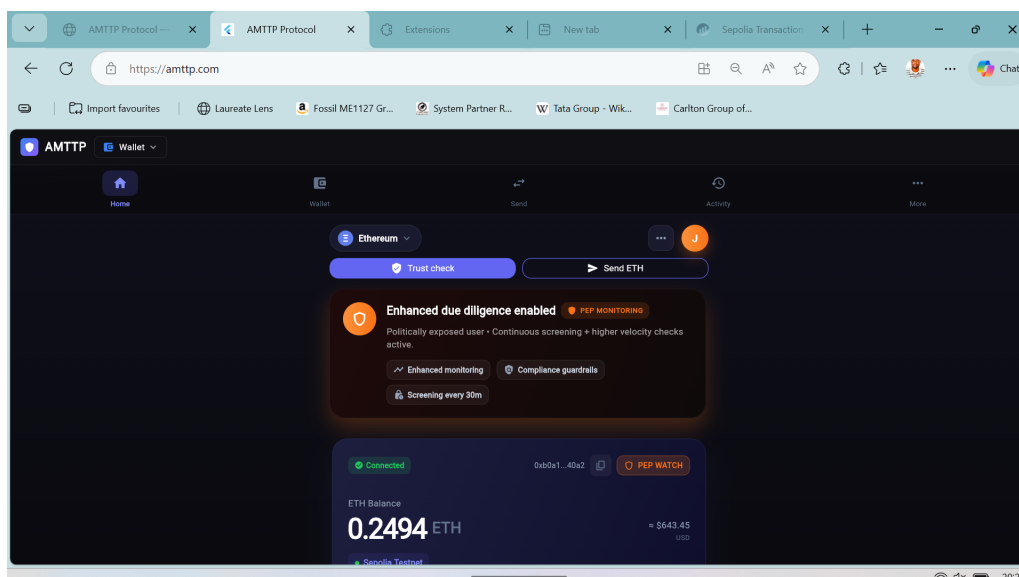


Figure 1: Live AMTTP consumer app at [amttp.com](https://amttp.com) — MetaMask wallet connected (0.2494 ETH on Sepolia), PEP Monitoring badge with Enhanced Due Diligence enabled, Trust Check and Send ETH actions. Real blockchain integration via raw `dart:js_util` interop.

## 2. Smart Contract Verification (On-Chain Finality)

Every consumer transaction passes through Solidity smart contracts deployed via **Hardhat** (`ethers.js v6`) using the **UUPS upgradeable proxy pattern (OpenZeppelin)**. The contracts enforce an `onlyOracle` access modifier, ensuring that only the trusted TypeScript Oracle Gateway can submit ML-validated risk scores to the blockchain.

**ZK-SNARK On-Chain Verification** (from `contracts/zknaf/sanctions_non_membership_verifier.sol`)

```
// Groth16 proof verification --- Sanctions Non-Membership
function verifyProof(
  uint[2] calldata _pA,    // Proof element A (G1 point)
  uint[2][2] calldata _pB, // Proof element B (G2 point)
  uint[2] calldata _pC,    // Proof element C (G1 point)
  uint[4] calldata _pubSignals // Public inputs
) public view returns (bool) {
  assembly {
    // BN254 G1 scalar multiplication via precompile 7
    g1_mulAccC(_pVk, IC1x, IC1y, calldataload(pubSignals))
    g1_mulAccC(_pVk, IC2x, IC2y, calldataload(add(pubSignals, 32)))
    // Pairing check via precompile 8: e(A,B)*e(vk_x,gamma)*e(C,delta)=1
    isOk := and(success, mload(_pPairing))
  }
}
```

Figure 2: Groth16 ZK-SNARK proof verification in Solidity — proves sanctions non-membership on-chain using BN254 elliptic curve pairings, without revealing the user's identity or transaction metadata.

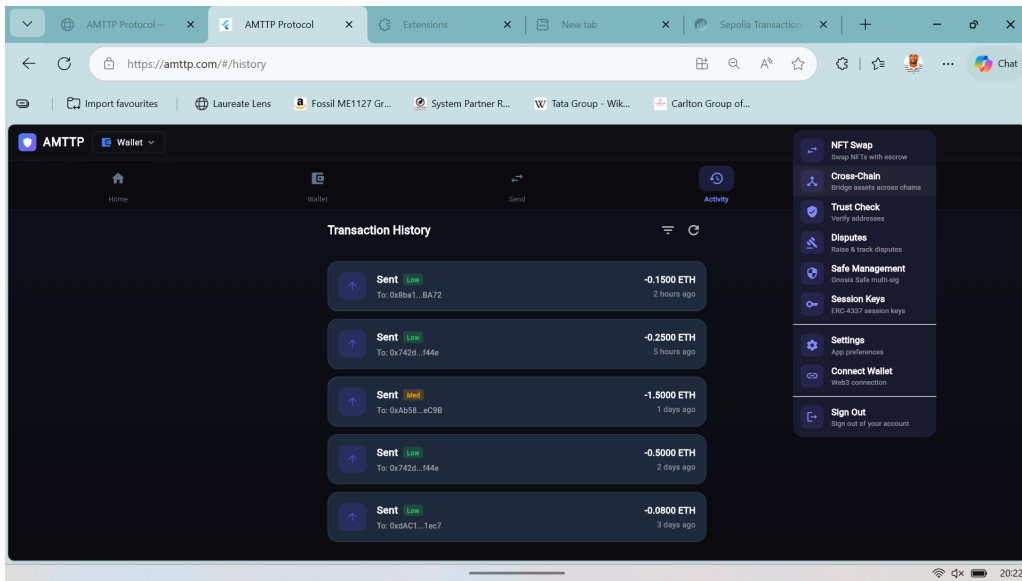


Figure 3: Transaction History with risk-level badges (Low/Med) and expanded navigation showing all consumer features — NFT Swap, Cross-Chain bridging, Trust Check, Disputes, Safe Management (Gnosis multi-sig), and ERC-4337 Session Keys.

### **3. End-to-End Proof of Concept**

The complete flow — from mobile app transaction request, through off-chain ML risk scoring, to on-chain ZK-proof validation and settlement — represents a differentiated architectural contribution to the Web3 compliance space. To the best of my knowledge, very few open-source or commercial products integrate all of these layers within a single independently built system.